

Chapter 11

DISTRIBUTED COORDINATION THROUGH ANARCHIC OPTIMIZATION

Stephen Fitzpatrick & Lambert Meertens

Kestrel Institute

3260 Hillview Avenue

Palo Alto, California

fitzpatrick@kestrel.edu & meertens@kestrel.edu

Abstract In this chapter, a *peer-to-peer* algorithm is described for *approximately* solving *distributed, real-time, constraint optimization* problems. The ANTS challenge problem is formulated as a distributed constraint optimization problem; an approximation version of the classical problem of graph k -coloring is formulated as a distributed constraint optimization problem to enable simple experimental assessment of the algorithm's performance.

Introduction

In this chapter, the ANTS challenge problem is formulated as a *distributed constraint optimization* problem (DCO) in which a set of distributed variables are to be assigned values that optimize some utility function, defined in terms of a set of variable-to-variable constraints.

A *peer-to-peer* (P2P) algorithm is introduced for approximately optimizing the variable assignment in soft real time; the algorithm is peer-to-peer in that every variable may be viewed as deciding for itself what value it should take and communicating that value to other variables with which it can directly interact.

The formulation of the challenge problem as a DCO is rather lengthy and detailed, so first DCOs are introduced using the simple, abstract problem of k -coloring the nodes of a graph. The P2P optimization algorithm is defined and its properties discussed using experimental results on k -coloring. Then the challenge problem is formulated as a DCO, the implementation of the P2P optimization algorithm for the challenge problem is detailed and some experimental results with the challenge problem simulator are presented.

1. Distributed Constraint Optimization

In a constraint optimization problem, variables v_i ($i = 1, \dots, n$) are to be assigned values from predetermined domains (which may be arbitrarily complex) such that the total value of a collection of *score functions* f_j ($j = 1, \dots, m$) is optimized (either maximized or minimized depending on the problem formulation). Each score function f_j is defined on a hyper-edge between some subset of the variables: a hyper-edge and a score function can together be considered a *constraint*. For simplicity of presentation, it is assumed that each score function has the real numbers as its range.

As an example, consider the classical problem of k -coloring a graph's nodes: each node is to be assigned one of k colors such that nodes connected by an edge have different colors. The *soft* version of this problem requires the number of *color conflicts*, i.e., the number of edges connecting nodes of the *same* color, be minimized. Note that in the soft version, the number of colors may be less than the graph's chromatic number (implying that it is impossible to eliminate all conflicts).

This problem can be formulated as a constraint optimization problem as follows. Each node's color is represented as a variable having domain $1, \dots, k$. Each edge in the graph has a *Dirac* score function δ_k (with domain $1, \dots, k$), where $\delta_k \equiv \lambda_{i,j} \cdot$ if $i = j$ then 1 else 0. Minimization of the total score function then corresponds to minimization of the number of color conflicts.

Constraint optimization problems are distributed when there is some inherent latency in the propagation of knowledge about changes in the variables' values. For example, the variables may represent the states of physically separated devices, for which knowledge about changes needs to be physically communicated.

Real-time considerations become important when the nodes or constraints may vary over time. For example, nodes representing physical devices may appear and disappear as those devices fail and recover. For the graph coloring example, it is assumed that each node/variable knows about the constraints in which it is involved; in the challenge problem, this knowledge will need to be explicitly maintained and is subject to some error and latency.

Real-time considerations become *critical* when the rate of change in the problem structure is commensurate with the communication latency (assuming that computational requirements are negligible). Then the problem is to continually compute variable assignments, so as to keep pace with the changing set of variables and/or constraints.

Under these conditions, the objective, strictly speaking, is to ensure that the variable assignment is *always* optimal; i.e., it is instantaneously re-optimized as the variables and/or constraints change. However, given a non-instantaneous communication latency, such an objective is likely unrealistic. A more reason-

able objective is to continually maintain a high-quality variable assignment; the overall performance of the algorithm can be measured in terms of the average over time of the total of the score functions.

1.1 Quality Metric: Degree of Conflict

For graph k -coloring, there is a simple metric for measuring the quality of an assignment of colors to the nodes. The *degree of conflict* γ is defined as the fraction of edges that are color conflicts. A γ of 0 corresponds to a proper (or perfect) coloring; a γ of 1 corresponds to a single-color coloring, in which every node has the same color and so every edge is a conflict.

If each node were randomly assigned one of k colors, then the expected value of γ would be $1/k$. This provides a useful baseline for assessing algorithm performance: an algorithm may be required to produce better-than-random results since a random coloring can be computed essentially instantaneously without communication.

Consequently, a normalized metric can be defined: $\Gamma \equiv k\gamma$. As before, a Γ of 0 corresponds to a proper coloring, but now a Γ of 1 corresponds to the expected quality of random solutions, and a Γ much greater than 1 indicates very poor solutions.

Similar metrics can be defined for other DCOs to allow comparison with random solutions.

2. A Peer-to-Peer Optimization Algorithm

The peer-to-peer optimization algorithm is an *ongoing, anytime* algorithm based on the principle of *local constraint optimization*. Periodically, each variable adjusts its own value so as to optimize the total values of those constraints in which it is involved, optimization being with respect to what it currently knows of the values of its neighbors (where two variables are neighbors iff there is a constraint in which they are both involved). And when a variable changes its value, it communicates the new value to its neighbors, thus enabling the preceding step to be performed without communication. (Note that the preceding step is assumed to execute in a critical section so that the variable's information about other variables cannot change during that step.)

The variables iterate this optimize-communicate cycle indefinitely, after random initialization. If the set of variables and/or constraints change, it is assumed (for now) that the variables become aware of the changes at the start of the optimization step and can therefore adjust their values to try to compensate for the changes. In other words, adaptivity is achieved through continual re-optimization.

If the set of variables and constraints stabilize, the variables *may* converge to a stable assignment of values. Such an assignment would be locally optimal (in

the hill-climbing sense, by which a change in any one of the values would reduce the overall quality); it typically is *not* globally optimal unless the problem is significantly under-constrained (i.e., there are many perfect solutions).

Moreover, if the variables change values frequently, then communication latency ensures that when one variable optimizes its own value, it is likely doing so with respect to values for its neighbors that are outdated, since at least some of its neighbors have likely changed value since the last time the variable received values from them.

Such *incoherence* may reduce the rate at which the variables converge to a stable assignment (assuming stable conditions). In extreme cases, incoherence may cause the assignment to become worse than random and cause the variables to *thrash*, i.e., to constantly change their values without improving the assignment quality.

Experiments [Fitzpatrick 2001] have shown that a simple way to reduce incoherence and avoid thrashing, at least for a wide range of graphs, is to introduce a stochastic element into when each variable optimizes its value: before optimizing its value, each variable generates a random number (in the range $[0, 1]$); if and only if this random number falls below some *activation probability* (also in the range $[0, 1]$) does the variable proceed with optimization (and subsequent communication if necessary). See Figure 11.1. Note that the choice of value in an optimization step may be non-deterministic and that this is a synchronous version of the algorithm, in which all of the variables operate in lockstep (an asynchronous version is considered later).

In the simplest algorithm, called the *Fixed Probability* algorithm (FP), the activation probability is fixed and is the same for each variable; thus, in Figure 11.1 $p_i = p$ for some constant p . It has been found experimentally that a value of around 0.3 works well. Setting the value high, say to 1.0, induces thrashing. Setting the value low, say to 0.1, avoids thrashing but reduces the rate at which the variables can adapt to changes in the set of variables and/or constraints (since adaptation occurs only when a variable changes its value).

How an optimal value is chosen by a variable will depend on the nature of the domain and constraints. For a small, discrete domain, exhaustive enumeration and evaluation of the choices may be efficient; for continuous or large domains, a hill-climbing algorithm may be used.

For graph coloring, a variable can compute a histogram of the colors used by its neighbors and select any color that corresponds to a minimum in the histogram. This method is particularly efficient for an ongoing algorithm since the histogram can be retained between optimization steps and incrementally updated only as the variable receives information from its neighbors indicated new values/colors.

Figure 11.2 illustrates typical behavior of FP under various activation probabilities (shown in parentheses beside each plot). It also illustrates why the

- Let C_i^s denote the value of node i in step $s \geq 0$.
- Each variable i randomly chooses an initial value C_i^0 from its domain, and informs its neighbors.
- The following synchronized loop is then performed indefinitely, for step $s = 1, 2, \dots$, simultaneously by each variable i :
 - 1 Determine an activation probability $p_i \in [0, 1]$.
 - 2 Generate a random number $r_i \in [0, 1]$.
 - 3 Choose a value C_i^{s+1} for the next step:
 - a. If $r_i < p_i$, choose C_i^{s+1} such that the total score for the constraints in which variable i is involved is optimized, given the current values of i 's neighbors.
 - b. If $r_i \geq p_i$, choose $C_i^{s+1} = C_i^s$ (i.e., no change).
 - 4 If $C_i^{s+1} \neq C_i^s$, inform neighbors.

Figure 11.1. Synchronous, stochastic peer-to-peer optimization

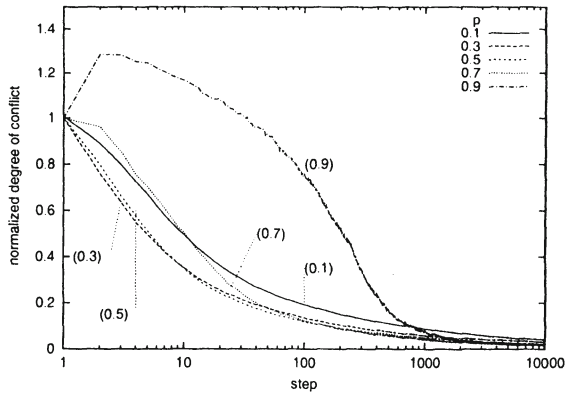


Figure 11.2. Effect of activation probability

algorithm is referred to as an anytime algorithm: assuming stable conditions and an appropriate activation probability, the algorithm can be allowed to run for an arbitrary time (shown on the horizontal axis) and it monotonically improves the solution quality (shown on the vertical axis as the normalized degree of conflict, so that low values indicate good solutions).¹

Anytime algorithms are particularly well suited to control problems: they avoid the need to make hard decisions about when to stop computing a solution and start implementing the solution, since a solution is always available.

¹Strictly speaking, a *performance profile* should be compiled for the algorithm to qualify it as an anytime algorithm. Such a profile allows a probability distribution over solution quality to be computed, given some characteristics of the variables and constraints and given a duration that the algorithm allowed to run.

Moreover, even as a solution is being implemented, the anytime algorithm can continue to improve upon the solution: solving and implementation can proceed in parallel.

2.1 Algorithmic Costs

To analyze the computational, storage and communication costs of the peer-to-peer algorithm, consider each line of Figure 11.1.

- 1 For the Fixed Probability version of the algorithm, determining an activation probability is vacuous and has essentially zero costs per variable.
- 2 Generating a random number has (presumably) constant computational and storage costs per variable, and no communication costs.
- 3 Deciding whether or not to activate is trivial, and if a variable does not activate, no costs are incurred. If a variable does activate, then the computational and storage costs will depend on the nature of the domain and the optimization process; for graph coloring, computing a color histogram has per-variable computational costs that are linear (at worst) in the number of neighbors and storage costs that are linear in the number of colors. Choosing an optimal color from the histogram has computational costs that are linear (at worst) in the number of colors. No communication takes place.
- 4 If a variable changes value, then the communication costs are at worst linear in the number of its neighbors (and the encoding requirements for the data being communicated). Using local, radio broadcasts, the communication costs may even be considered to be independent of the number of neighbors.

The main result of this analysis is that the per-step costs for a given variable are determined by the number of its neighbors rather than by the total number of variables. Thus, if the mean number of neighbors per variable is bounded as the number of variables grows, the per-variable costs will also be bounded; in other words, the algorithm is scalable, in principle, to arbitrarily large systems.

(Note that for some common types of graphs, such as complete graphs, the mean number of neighbors per variable is *not* bounded as the number of variables increases. However, it seems unlikely that such graphs will be relevant for realistic problems: technological limitations would likely preclude systems containing millions of variables in which each variable has to interact with every other variable.)

2.2 Experimental Results

Experiments have been performed on the peer-to-peer algorithm for k -coloring graphs having sizes ranging from a few hundred to a few thousand nodes. Several types of graph have been used:

- Regular 2-dimensional grids with nodes at integral positions along the x - and y -axes and with edges connecting immediate neighbors along each axis (with and without wraparound at the edges). These graphs have mean degree 4 (for infinite graphs in the non-wraparound case) and chromatic number 2.
- Regular 2-dimensional grids with nodes at integral positions along the x - and y -axes and with edges connecting immediate neighbors along each axis *and along each diagonal* (with and without wraparound at the edges). These graphs have mean degree 8 (for infinite graphs in the non-wraparound case) and chromatic number 4.
- Regular 3-dimensional grids with axial and diagonal edges. These graphs have mean degree 26 (for infinite graphs in the non-wraparound case) and chromatic number 8.
- Random graphs with given size, mean degree and chromatic number. To construct such a graph, the nodes are partitioned in χ classes, where χ is the specified chromatic number, and the required number of edges connected at random between nodes in different classes. The true chromatic number of the generated graph is not higher than the specified chromatic number and for graphs that are large and have a mean degree that is not too small, the true chromatic number is likely to be exactly the specified chromatic number. It should be noted that, especially for high mean degrees, this method of construction is not guaranteed to produce a random sampling of the space of graphs having the specified properties.

Using such graphs, experimental evidence has been gathered to support the following conclusions.

An activation probability of around 0.3 produces rapid improvement while avoiding thrashing. It is not clear if or why 0.3 is a good value for *all* graphs: it can easily be imagined that very dense graphs would require lower values, since the likelihood of a variable changing value at the same time as a neighbor increases with the density. Nor is it clear that the same value should be used for the entire graph: graphs with highly varying density, for example, may benefit from a non-uniform activation probability. Nevertheless, the experimental evidence suggests that 0.3 is a good value for a wide range of graphs, and it is assumed for the remaining results.

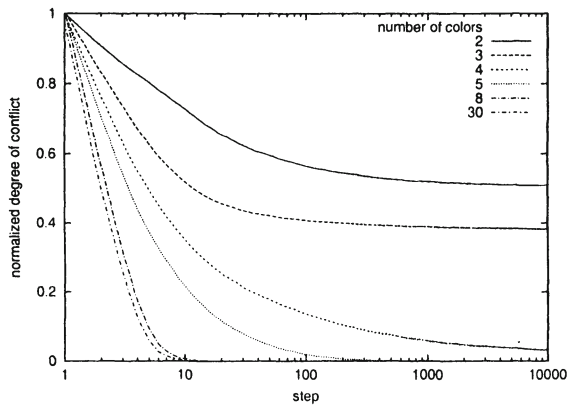


Figure 11.3. Effect of number of colors; chromatic number=4

The algorithm tends to asymptotically approach an optimal assignment of values in *over-constrained* and *critically constrained* problems and tends to quickly produce truly optimal assignments in significantly *under-constrained* problems, where the degree of constraint is defined according to whether the number of colors is less than, equal to or greater than the chromatic number. For the regular grids, it is possible to compute the highest quality coloring that can be achieved, and experiments show that the algorithm typically produces colorings within a few percent of optimal.

For very under-constrained problems, the simple algorithm shown in Figure 11.1 actually performs quite poorly: because the problem is very under-constrained, there are many colors that are optimal in each optimization step; because the variables choose randomly from amongst the optimal colors, occasionally neighbors choose the same color by chance. This problem is readily overcome by prohibiting a variable from activating if it has no conflicts with its neighbors. The performance of this algorithm is illustrated in Figure 11.3.

The algorithm is scalable in performance as well as costs. See Figure 11.4. It was argued above that the algorithm has per-variable, per-step costs that are determined the density of the graph rather than its total size: this conclusion is supported by experiments. Moreover, experiments show that the *quality of coloring* versus number of algorithm steps is (statistically) independent of the size of the graph.

The algorithm is robust against communication error. Communication error can be simulated by randomly corrupting or dropping messages between variables. Experiments show that the degradation in performance increases with the degree of disruption, but that catastrophic failure does not result. For example, Figure 11.5 shows the effect of various rates of message loss.

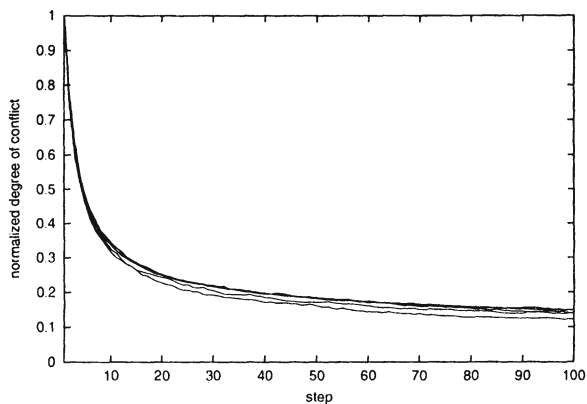


Figure 11.4. Effect of graph size

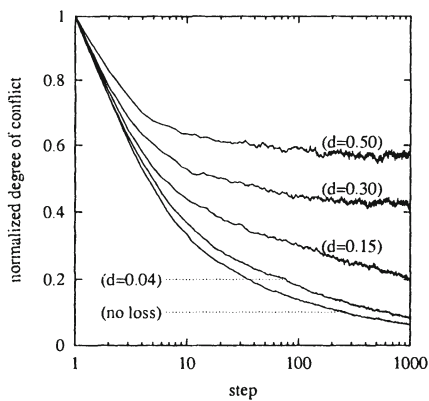


Figure 11.5. Effect of message loss

The algorithm is robust against topological changes. The topologies of the graphs can be randomly changed by periodically removing some nodes and reinserting some previously-removed nodes (along with incident edges) to maintain a constant fraction of nodes removed. If the fraction of removed nodes is small, then it is likely that the structure of the graph is unchanged by such turnovers; this is important for experiments as changing, say, the graph's chromatic number may in itself significantly affect the algorithm's performance.

If the period is short (say, 1 step) then small rates of turnover have little impact on the performance of the algorithm — see Figure 11.6. If the period is long (say, 30 steps) then moderately high rates of turnover are also tolerable: for example, Figure 11.7 shows that the quality of the coloring degrades

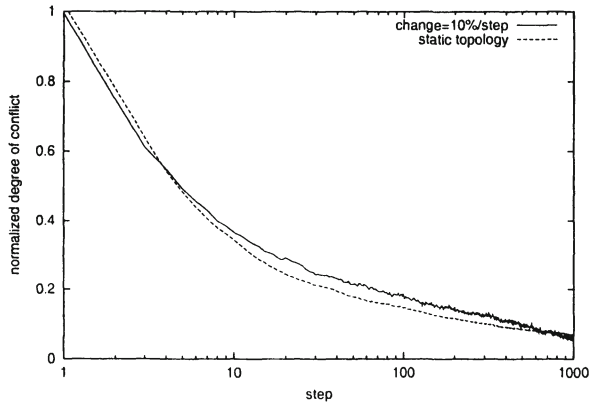


Figure 11.6. Effect of continual, minor change

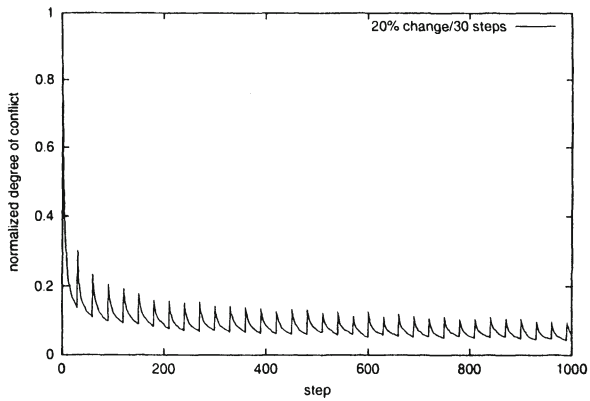


Figure 11.7. Effect of intermittent, major change

immediately following a turnover, but quickly recovers and overall the quality continues to improve.

2.3 Asynchronous Execution

Although the preceding sections discussed a synchronous version of the algorithm, asynchronous execution is feasible provided that the (mean) rate of change is low enough to allow information to be propagated in spite of the communication latency, as shown by the following experiment.

Assume that each variable is *awoken* with a uniform time period that can be taken to be the unit of time, and that the wake-up times of the nodes are randomly distributed through one period. When a node is awoken, it determines whether or not to activate, and if it does activate it optimizes its color, and if

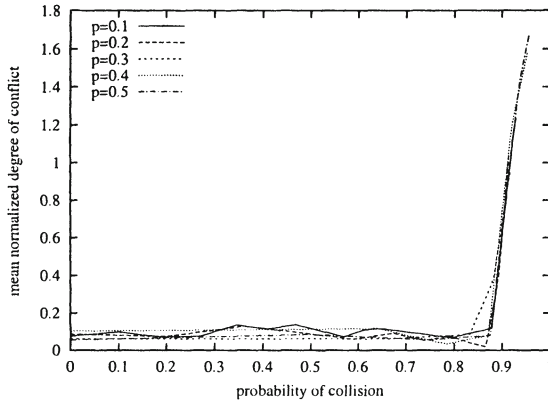


Figure 11.8. $\bar{\Gamma}$ vs. $1 - (1 - p)^L$

its color changes it communicates its new color to its neighbors, exactly as in the synchronous algorithm; after this process, a variable sleeps until awoken again.

Let the communication latency (measured in units of the wake-up period) be L . Then preliminary experiments suggest that the communication latency has little effect on the performance of the algorithm provided that the following condition holds: $1 - (1 - p)^L < P_T$ where p is the activation probability and the *threshold* P_T is a constant for a given graph, and perhaps for a class of graphs. Above the threshold, the performance of the algorithm *rapidly* degrades, leading to thrashing. See Figure 11.8.

Intuitively, the term $1 - (1 - p)^L$ measures the probability that a variable will change color while a color-change message is inbound from another variable. Provided that this probability is small, then the distributed nature of the variables is unimportant. The surprising result is that the performance degrades so sharply above the threshold, which suggests that some form of phase change may be taking place (see Chapter 13).

Further experiments suggest that very high latencies causes the coordination mechanism to be caught in an out-of-phase, oscillating trajectory. The period of oscillation appears to be at least twice the latency, corresponding to a phase lag of less than π . This oscillation is observed as almost all variables simultaneously assume each of the colors in turn, resulting in very poor quality.

3. Radar Tracking

In the preceding section, the distributed coordination algorithm was defined and its use and performance illustrated with the abstract problem of graph k -coloring. In this section, the ANTS challenge problem is formulated as a dis-

tributed constraint optimization problem and a solution presented in the form of the distributed optimization algorithm.

Coordination in the challenge problem is based on *scheduling* the actions of the sensors: each sensor operates a continual process that determines ahead of time what measurements it is going to take, and when. It communicates its schedule to other, nearby sensors, which take the schedule into account when determining their own schedules.

In order to produce a schedule, a sensor uses a quality metric that assesses how well a proposed set of measurements (to be taken by itself and other sensors) will scan known targets, based on the expected behavior of those targets. The objective of the coordination mechanism is, essentially, to optimize this metric through the entire sensor network.

In the following sections, after a summary of the challenge problem, the basis for reasoning about expected target behavior, *world estimates*, is introduced. Then a metric on sensor schedules, with respect to world estimates, is defined. Initially, no consideration is given to such practical concerns as the distributed nature of the sensor network and communication latency; these concerns are addressed by showing how world estimates and sensor schedules can be distributed. Then other practical issues relating to the implementation of the coordination mechanism for the challenge problem are discussed. Finally, some experimental results are presented.

The analogy between distributed graph coloring and distributed sensor coordination should be clear: in each case, each variable (node or sensor) is responsible for determining its own value (color or schedule); when determining its own value, each variable optimizes some metric defined on local knowledge (local copies of neighbors' colors or local copies of neighbors' schedules and local world estimates); moreover, the *process* of distributed optimization is stabilized without inter-variable communication (thrashing being avoided using local, stochastic techniques).

Moreover, in both cases, the extent of inter-sensor coordination is limited by an essential locality to the problem: by discrete edges in graph coloring and by the limited scanning range in the challenge problem. This limitation is essential to scalability.

3.1 Summary of the Challenge Problem

The details of the challenge problem are presented elsewhere in this collection — see Chapter 2. Here, the pertinent features are summarized.

Each radar sensor consists of three emitter/detector pairs, oriented at 120° intervals, only one of which can be scanned at any given time, with Each scan taking up to 0.6 seconds for an amplitude-only measurement or 1.8 seconds for an amplitude-and-frequency measurement.

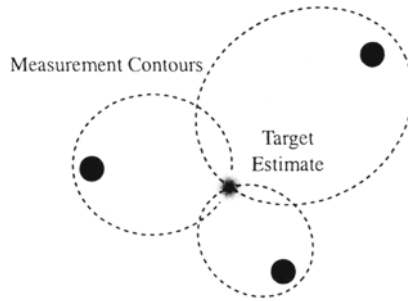


Figure 11.9. Determining a target's position using trilateration

Each emitter can be independently activated or deactivated. While an emitter is active, it consumes power, regardless of whether or not its detector is being sampled. If a deactivated emitter is activated, the emitted beam is unstable and does not give reliable measurements for approximately 2 seconds.

The strength of the signal reflected off a target is determined by the following equation:

$$m(R, \theta) = K e^{-(\theta/A)^2} / R^2 \quad (11.1)$$

where R is the sensor-target distance, θ is the angle between the emitter's mid-beam and the target ($0^\circ \leq \theta < 180^\circ$), and K and A are constants. The constant A is about 40° so that, for example, a target that is at mid-beam will produce a signal that is $e^{9/4} \approx 9.5$ times larger than a target that is at the same distance from the sensor but that is 60° off mid-beam.

The emitters, detectors and sampler are subject to random noise (both internal and environmental) that limits the effective range of the sensors to about 7 meters.

If a radar beam reflects off several targets simultaneously, the reading produced by the detector is essentially a random combination of what the individual targets would produce separately, and is effectively useless.

Each radar sensor has an omni-directional radio transmitter and receiver that can be used to communicate measurements and arbitrary data in short bursts (message lengths should preferably be kept below 100 bytes). The communication range is limited by the effective physical transmission range, so its 'broadcast' mode is really a local, multicast mode.

To accurately locate a target, more-or-less simultaneous measurements from several, nearby sensors must be combined using *trilateration*, as shown in Figure 11.9. Of course, readings from sensors are subject to noise so new measurements for a target are combined with the target's *track* to achieve the best compromise between where the target is expected to be (based on its history) and where the sensors report it to be.

This chapter does not go into the details of the tracking/data fusion algorithm. Rather, it is assumed that one of the objectives of the coordination mechanism is to achieve simultaneous scans of each target and that estimates of target positions can be obtained by extrapolating tracks.

The number of sensors may be large and the number of targets and the availability of sensors may vary over time, so the coordination mechanism must be scalable, adaptive and robust.

In the following sections, the coordination necessary to achieve good tracking is formulated as a DCO and the details of applying the distributed optimization algorithm are presented.

3.2 World Estimates

The objective of a sensor network is to cost-effectively maintain an estimate of certain properties of the real world. For example, for a single target, a network's estimate may include the target's position, velocity, acceleration, facing (i.e., orientation relative to velocity), etc. The cost of operating the network may be measured as the amount of battery energy consumed, for example.

The following assumption, although not strictly necessary for the abstract formulation developed in this section, is useful for pedagogic purposes:

ASSUMPTION 1 (SEPARABLE TARGETS) *A world state is a finite map from some uninterpreted domain of targets to single-target information.*

In other words, the world is assumed to be made up of separately identifiable targets, for each of which independent information is represented. The set of targets for which information is represented by world state w is denoted by $\text{dom}(w)$ and the information for target g is denoted by $w(g)$.

The notation $e \equiv \{x \in X \rightarrow y\}$ is used to indicate that e is a finite map whose domain is X and that each x in the domain has corresponding value y (which may depend on x). Thus, $w \equiv \{g \in \text{dom}(w) \rightarrow w(g)\}$.

Typically, the state of the real world is not known precisely. A *world estimate* may be mathematically expressed by viewing the “true” real-world state as a random variable with a probability distribution over the space of world states. For convenience we assume that world states consist of continuous components (e.g., position and velocity), and that an estimate is continuous and may be given by a probability *density* function. (In general, world states may be discrete or hybrid, and the probability distribution could be any *measure* whose total is one, but note that such distributions may be approximated or may be represented by densities like Dirac functions.) If ω is the density function for some probability distribution over the space of world states W , then $\omega(w)$ is the probability density that the real world has state w (where $w \in W$).

The *quality* of an estimate quantifies, in some way, the accuracy of the knowledge conveyed, where high quality corresponds to high accuracy. For example,

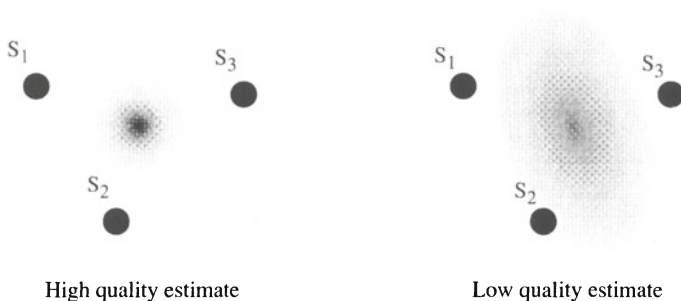


Figure 11.10. Quality of a position estimate reflects how accurately the position is known

the quality of an estimate of a single target's position might be stated in terms of the variance of the position's probability density (see Figure 11.10). In general, assume that the quality of an estimate is quantified as a real number and let it be denoted by $\zeta(\omega)$.

ASSUMPTION 2 (MODEL OF WORLD EVOLUTION IS GIVEN)

It is assumed that an estimate ω of the real world's state at some time t_0 can be projected to give an estimate $\epsilon_{t_0 t} \omega$ at another time $t > t_0$ according to a model of world evolution ϵ .

For simplicity of notation, the subscript ' t_0 ' is omitted from such expressions as $\epsilon_{t_0 t} \omega$ since evolution is always relative to the time for which the estimate holds (which can be assumed to be represented as part of the estimate itself).

For example, suppose that a target is modeled as having a constant velocity and a constant velocity probability distribution, and that a world state represents the target's position and velocity: $w \equiv \langle p, v \rangle$. Then an estimate can be projected as follows.

- Let $\text{vel}(\omega)$ denote the density function of the probability distribution for velocity, regardless of position: $\text{vel}(\omega) \equiv \int \omega \langle p, v \rangle dp$. This distribution does not change: $\text{vel}(\epsilon_t \omega) = \text{vel}(\omega)$.
- The mean of the position changes according to the usual linear model: $\bar{p}_t = \bar{p}_{t_0} + \bar{v}_{t_0}(t - t_0)$, where $\bar{p} \equiv \int \int p \cdot \omega \langle p, v \rangle dp dv$ and $\bar{v} \equiv \int \int v \cdot \omega \langle p, v \rangle dp dv$.

However, the uncertainty in the initial position estimate is augmented by the uncertainty in the velocity estimate, magnified over time. More precisely, since the velocity is known to be constant:

$$\epsilon_t \omega \langle p, v \rangle = \omega \langle p - v(t - t_0), v \rangle .$$

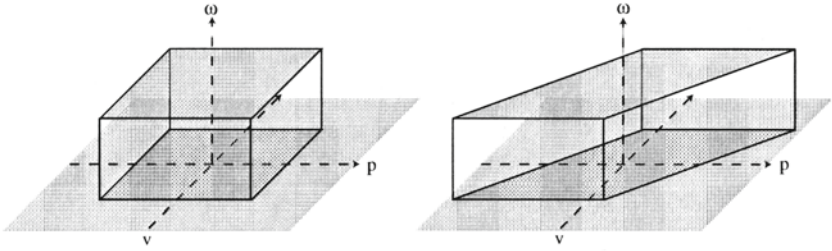


Figure 11.11. Projection of a uniform probability distribution over positions and velocities

For example, Figure 11.11 (left) shows (the density for) a uniform probability distribution over positions and velocities in some range. Projecting this probability distribution causes each constant-velocity plane to be translated by an amount that is proportional to the velocity (in particular, the plane at $v = 0$ is unmoved). The resulting probability density is shown in Figure 11.11 (right). The total range of positions with non-zero probability density, and thus the uncertainty in the position, has increased.

In general, models of evolution are more complex; for example, generally no component is known exactly, several of the components may be mutually dependent, and evolution may be non-deterministic.

3.3 Trajectories

A *discrete world trajectory* (or just a *trajectory* for brevity) on an n -vector of times $\vec{t}$ (where $t_1 < t_2 < \dots < t_n$) is an n -vector of world states $\vec{w}$ and has the meaning that the world has state w_i at time t_i .

For example, a simple trajectory, for a world in which only a single target is present, may be comprised of position and velocity coordinates at various times. In general, each element of a trajectory is a complete world state and may represent information about multiple targets.

The *a priori* probability distribution of a trajectory occurring, according to some initial world estimate ω and some model of evolution ϵ , can be computed as follows:

- The initial world estimate ω is projected to the time t_1 of the first world state: $\omega_1 \equiv \epsilon_{t_1} \omega$. Then the probability density of the first world state w_1 occurring is $p_1 \equiv \omega_1(w_1)$.
- The first world state is then projected to the time of the second world state to determine a conditional estimate ω_2 that gives the probability density of a world state occurring, given that w_1 occurs. A world state w can be projected by first lifting it to a single-point estimate $\hat{w}$ (which

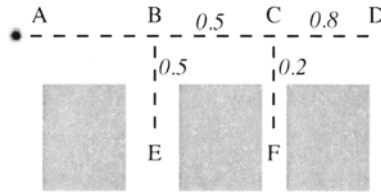


Figure 11.12. Possible trajectories through an environment of obstacles

has a Dirac probability density function that is everywhere zero except at some specified point, where the density is infinite) and projecting as usual: $\omega_2 \equiv \epsilon_{t_2} \hat{w}_1$. Thus, the probability density of w_2 is $p_2 \equiv \omega_2(w_2)$.

- The probability density of the third world state is determined by projecting the single-point estimate of the second world state: $p_3 \equiv \omega_3(w_3)$ where $\omega_3 \equiv \epsilon_{t_3} \hat{w}_2$. And so on.
- The overall probability density of the trajectory is the product of the individual probability densities.

Formally, a model of evolution ϵ induces a probability distribution with density function $\Omega_{\omega\epsilon}$ over the space of trajectories $\vec{W} \equiv \prod_{i \in \text{dom}(\vec{t})} W$ on a vector of times $\vec{t}$, from an initial estimate ω over W , where

$$\Omega_{\omega\epsilon}(\vec{w}) \equiv \prod_{i \in \text{dom}(\vec{t})} \omega_i(w_i) \quad (11.2)$$

where

$$\omega_i \equiv \begin{cases} \epsilon_{t_1} \omega & \text{if } i = 1, \\ \epsilon_{t_i} \hat{w}_{i-1} & \text{if } i > 1. \end{cases} \quad (11.3)$$

For simplicity of notation, the subscripts on $\Omega_{\omega\epsilon}$ are omitted — it should be clear from context which estimate and which model of evolution pertain.

Figure 11.12 illustrates a simple example, in which a single target is moving through a grid of obstacles (for example, a ground vehicle moving along streets). The target starts at point A; at point B, the target has equal probabilities of continuing in a straight line to C, or making a right-angle turn to E; at point C, the target has a probability of 80% of continuing to point D, and 20% of turning to point F.

Assuming that only the target's position is of interest, the possible trajectories and associated probabilities are $p(\text{ABE}) = 0.5$, $p(\text{ABCF}) = 0.5 \times 0.2 = 0.1$, and $p(\text{ABCD}) = 0.5 \times 0.8 = 0.4$. Note that trajectories such as ABEC and ABCDF are infeasible.

3.4 Measurements and Sensor Models

Because sensors are based on physical processes that are subject to noise, there is typically some uncertainty associated with *observations* or *readings*, which are the results of taking measurements. For example, if a target has a weight of 5.000 units, a sensor may report the weight as a random variable with log-normal probability distribution having mean 5.01 and variance 0.1. Let $m(w)$ denote the results of taking a measurement m when the real world has state w .

ASSUMPTION 3 (SENSOR MODELS GIVEN)

A *sensor model* ρ_{wm} is the *a priori* probability density function over the range R of sensor readings: $\rho_{wm}(r)$ is the probability density that reading $r \in R$ will be obtained for measurement m when the real world has state w .

It is assumed that there is a quality metric on proposed measurements $\phi_w(m)$ that reflects the expected quality of information obtained. For example, in an informal sense, a radar is expected to produce higher quality measurements for close targets compared with distant targets. A state may represent multiple targets and a measurement may acquire information regarding any subset of the targets, so it is assumed that the quality metric can be represented as a map over targets:

$$\phi_w(m) \equiv \{g \in \text{dom}(w) \rightarrow \phi_{wg}(m)\} . \quad (11.4)$$

For the radar example, the basis of the measurement quality metric *for a single target* is Equation 11.1 which predicts how strong a signal should be obtained from a target. However, the quality metric should probably not be *directly* proportional to the signal strength: an example is given in Equation 11.13.

When multiple targets are present, interference may occur; i.e., the readings acquired by a sensor may not be a simple union of the readings that would be received for each target if it alone were present. A simple example of interference is one target blocking another target from the view of a line-of-sight sensor.

For *non-interfering targets*, $\phi_{wg}(m)$ can be defined for an individual target without regard to other targets; i.e., $\phi_{wg}(m) \equiv \phi_{w(g)}(m)$ where $\phi_{w(g)}(m)$ denotes the quality of a measurement for a single target in the absence of other targets.

For the radar example, interference can be accounted for by the following formula:

$$\phi_{wg}(m) \equiv \max \left[0, \phi_{w(g)}(m) - \sum_{g' \in \text{dom}(w) \setminus \{g\}} \phi_{w(g')}(m) \right] . \quad (11.5)$$

- In the case where one signal is much stronger than the others: for the target that produces the strongest signal, the left $\phi_{w(g)}$ term will dominate and the resulting quality will be almost the same as if only this target were present; for the other targets, the sum term will dominate and the overall quality metric will be zero.
- In the case where there are several strong signals, the sum term will always be larger than or comparable to the left $\phi_{w(g)}$ term and the quality metric for every target will be low.

3.5 Data Fusion

Readings obtained from sensors are fed to a *data fusion processor* that computes estimates of the real world from the readings. In the challenge problem, the data fusion processor is a multi-target tracker, which may be based on the single-target trackers described in Chapters 3 and 5.

Informally, an *on-line* or *real-time* data fusion processor D may be characterized by the equation $\omega' = D_\omega \vec{r}$ where ω' is a new estimate computed from an existing estimate ω and a vector of readings $\vec{r}$ acquired by measurements $\vec{m}$ at times $\vec{t}$.

Given a *proposed* vector of n measurements, let $\vec{R}$ denote the space of possible vectors of readings: $\vec{R} \equiv \prod_{i=1, \dots, n} R_i$ where R_i is the space of possible readings for the i^{th} proposed measurement.

Let $P_{\omega \vec{m}}$ denote the density function of the probability distribution for the vectors of readings. This probability density is determined by the initial world estimate ω , the model of evolution ϵ and the sensor models ρ_i :

- The initial estimate and the model of evolution determine a probability density function Ω over possible trajectories on $\vec{t}$.
- For each trajectory, the probability density for the possible vectors of sensor readings is determined from the sensor models and the individual world states that comprise the trajectory.
- The overall probability density for a vector of readings is a weighted sum of the single-trajectory probability densities.

Given $P_{\omega \vec{m}}$, the *expected* quality of the new estimate is

$$[[\zeta(\omega')]] = \int_{\vec{R}} P_{\omega \vec{m}}(\vec{r}) \cdot \zeta(D_\omega \vec{r}) d\vec{r}. \quad (11.6)$$

3.6 Coordination Mechanism

The following assumption is made:

ASSUMPTION 4 (UNCOUPLED SENSOR NETWORK AND TARGETS) *The actions of the sensor network do not affect the evolution of the world state.*

There are two particular implications of this assumption that should be noted:

- The coordination mechanism cannot direct the targets of observation to improve measurement. This implication fits better with sensor networks observing adversaries, rather than, say, a manufacturing plant.
- The targets of observation do not change behavior as a result of being measured. It is in general possible that a target may detect that it is being illuminated by a radar, for example, and change course or attempt to jam the radar. In this treatment, such possibilities are ignored.

Under this assumption, the responsibility of a real-time coordination mechanism may be phrased as follows: determine a vector of measurements, to be taken over some reasonable (short) time span, that optimizes the trade-off between the expected quality of the next estimate and the cost of taking the measurements.

There are two general techniques that might be used to determine an optimal vector of measurements:

- **Analysis** — Given a model of evolution, sensor models, and a precise definition of the data fusion process, it may be possible to devise a simple algorithm that will directly determine optimal measurements for maximizing the quality of estimates and minimizing the cost of operation.
- **Search** — A finite approximation of the space of all feasible vectors of measurements can be exhaustively searched. For each vector of measurements, the initial world estimate, the model of evolution and the sensor models can be used to determine a probability distribution for vectors of readings; each vector of readings can be given to the data fusion processor and the quality of the resulting estimate computed; then the expected quality can be computed from the probability density function of readings for this vector of measurements. The coordination mechanism can use these predictions of quality to determine which vector of measurements optimizes the quality-cost trade-off. See Figure 11.13.

However, analysis is likely to be intractable for non-trivial sensor networks, and while search is theoretically possible for any arbitrary network, its computational costs are almost certainly prohibitive:

- 1 The space of feasible vectors of measurements is typically large.
- 2 The space of possible readings for each vector of measurements is typically large.

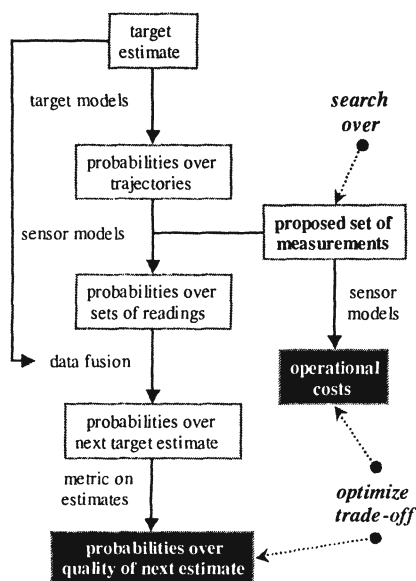


Figure 11.13. Optimization of trade-off between expected quality of estimates and operational costs

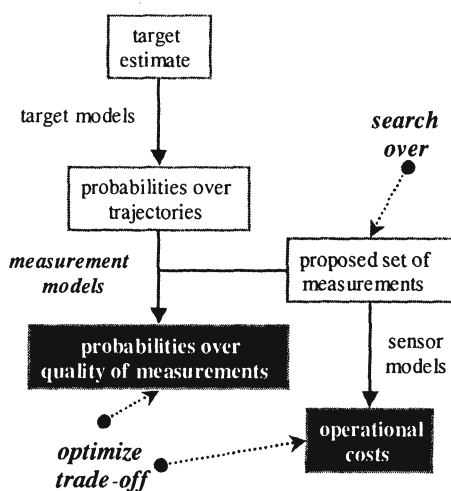


Figure 11.14. Optimization of trade-off between expected quality of measurements and operational costs

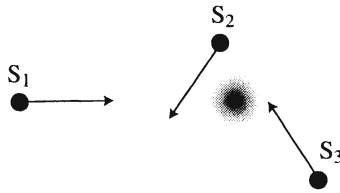


Figure 11.15. Distance and angle between target and emitter form the basis of a proximate metric

- 3 Computing the probability density of vectors of readings for a given vector of measurements is typically computationally expensive when done precisely, as it involves numerous convolutions.
- 4 Computing a new world estimate for each possible vector of readings is computationally expensive.

The combination of large search spaces and computationally expensive processes at each node of the search space typically make the search approach, in the form stated above, infeasible for real-time systems. An alternative is considered below.

3.7 Proximate Metric

As explained above, coordination based directly on maximizing the expected quality of world estimates is typically infeasible. However, the computational requirements of the search approach can be significantly reduced by using a *proximate metric* that directly gauges the quality of measurements without computing new world estimates — see Figure 11.14. It thus eliminates the costs associated with items 2, 3 and 4 above, although it does introduce other, lower costs.

Conceptually, optimizing with respect to the proximate metric ensures that the sensors acquire data that is *likely* to lead to high quality world estimates. For example, Figure 11.15 shows three radars scanning a target that has a high-quality position estimate. An admittedly simplistic proximate metric, based on Equation 11.1 might award high scores to measurements made by sensor S_3 because, using the indicated emitter-detector, it has a good combination of distance and angle. In contrast, measurements from S_1 would achieve low scores because the distance is too large, and measurements from S_2 would achieve moderate scores because, although the distance is reasonable, the angle is high.

A more realistic proximate metric needs to account for how multiple measurements complement each other (e.g., tracking in the radar example requires

simultaneous measurements from multiple sensors) and how well complete trajectories (not just single positions/world states) are scanned.

Of course, in many cases, given the difficulty of analyzing data fusion processes and the need for low computational costs, the proximate metric will be heuristic. Even so, it is likely that the coordination mechanism will need to greatly reduce the size of the search space to achieve real-time performance. But since the function of the proximate metric is to guide sensor coordination rather than, say, directly guide counter-measures, a lack of rigorous fidelity is likely to be acceptable.

In the remainder of this section, the proximate metric is considered in abstract terms — no regard is given to computational costs.

3.8 Proximate Metric with respect to Probability Distributions over Trajectories

A proximate metric Ψ measures the quality of a proposed vector of measurements $\vec{m}$, to be taken at times $\vec{t}$, given an initial world estimate ω and a model of evolution ϵ . Conceptually, Ψ is based on considering how well the proposed measurements suit possible trajectories, and forming some cumulative assessment for the probability density function Ω of possible trajectories on $\vec{t}$ induced by ϵ from ω . For example:

- Ψ may be based on a weighted sum over the probability density:

$$\Psi_{\Omega}(\vec{m}) \equiv \int_{\vec{w}} \Omega(\vec{w}) \psi_{\vec{w}}(\vec{m}) d\vec{w} \quad (11.7)$$

where $\psi_{\vec{w}}(\vec{m})$ gauges the quality of a vector of measurements with respect to a single trajectory $\vec{w}$.

- Ψ may be based on how well the proposed measurements suit the most likely trajectory $\vec{w}^*$:

$$\Psi_{\Omega}(\vec{m}) \equiv \psi_{\vec{w}^*}(\vec{m}) \cdot \Omega(\vec{w}^*) . \quad (11.8)$$

While this form of metric is presumably less reliable than the weighted-sum form, it may be useful for simple target models because its computation may be much less expensive. This form will be used below in the radar example. (The quality metric includes a factor, $\Omega(\vec{w}^*)$, for how likely is the most likely trajectory because a trajectory that is highly likely to occur may warrant greater expenditure by the sensor network than a trajectory that is unlikely to occur.)

Regardless of which form is chosen, the basis of the metric is the single-trajectory metric $\psi_{\vec{w}}(\vec{m})$ — this is considered in the next section.

3.9 Quality of Measurements with respect to Single Trajectory

Given a trajectory $\vec{w}$ of world states at times $\vec{t}$, the quality $\psi_{\vec{w}}(\vec{m})$ of a proposed vector of measurements $\vec{m}$ (also at times $\vec{t}$) is a function of two terms: (i) the quality of data acquired by each measurement of its corresponding world state, and (ii) how well the measurements complement each other.

In general, these two terms are highly application-specific — they essentially are heuristics that attempt to characterize the type of sensing that is likely to lead to good tracking. Here, forms appropriate for the radar example are considered.

Section 11.3.4 refines term (i) into a map from individual targets in the world state to single-measurement, single-target metrics $\phi_w(m) \equiv \{g \rightarrow \phi_{wg}(m)\}$ and gives a form for $\phi_{wg}(m)$ that accounts for the possibility of target interference.

The form of term (ii) is based on the requirement that two or three measurements from different sensors be taken approximately simultaneously, which can be accommodated using a combination of two functions:

- persistence — a function that associates a measurement with the period of time over which it could usefully be combined with other measurements;
- adhesion — a function that, for a given time t , computes the combined quality that arises from all measurements whose persistence functions indicate that they can usefully contribute at time t .

These two functions and their combination are detailed below.

3.9.1 Persistence. A measurement's persistence function relates the measurement's *effective* quality at any arbitrary time to the measurement's peak quality (which is associated with the mid-point of the period over which the measurement was taken). A persistence function can have arbitrary form, but typically it drops off monotonically (to zero) with distance from the measurement's mid-point. For example, Figure 11.16 shows a 'triangular' persistence function.²

Specifically, for a measurement whose mid-point is at time t_0 and whose quality at t_0 is ϕ_0 , the *effective* quality at time t is given by

$$\phi_t \equiv \phi_0 \pi(t - t_0) \quad (11.9)$$

²'Persistence' is somewhat a misnomer since there is a period *before* the measurement is performed for which the function is non-zero.

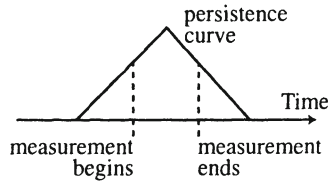


Figure 11.16. Persistence of a measurement

where the persistence function π has range $[0, 1]$. Note that ϕ_0 is a map from targets to single-target quality metrics whereas $\pi(t - t_0)$ is a scalar, so computing their product involves scaling each single-target metric in ϕ_0 uniformly, as detailed in Section 11.3.9.4.

3.9.2 Adhesion. The persistence function effectively smears out a measurement over a non-instantaneous time period. For a given instant, the overall quality of measurement results from the combination of all measurements that have been smeared onto that instant: this is quantified as an adhesion function α that takes as argument a vector $\vec{\phi}$ of simultaneous, single-measurement quality metrics and computes a combined metric.

For the radar example, an appropriate adhesion function for a single target is shown in Figure 11.17:

- When the sum of the metrics for single-measurements is around two or three, the adhesion function awards a high overall quality.
- When the sum of the individual quality metrics is less than 1, the adhesion function awards a low overall quality. One consequence of this is that, if the coordination mechanism can arrange for more sensors to scan the target, it achieves a high *payoff* because the increase in overall quality is high compared with the increase in sensing costs.
- For higher sums, the overall quality awarded is also higher (the function is monotonic) but the rate of improvement rapidly decreases and the function asymptotically approaches 1. One consequence of this is that the coordination mechanism is discouraged from swamping a single target with many sensors, since the increase in quality would be low compared with the increase in sensing costs.

This adhesion function captures the informally-stated preference for two or three simultaneous measurements. However, it should be noted that the requirement is really for multiple measurement from *different* sensors. The persistence functions for measurements from the same sensor may be combined, e.g., by taking their maximum at each instant, as illustrated in Figure 11.18.

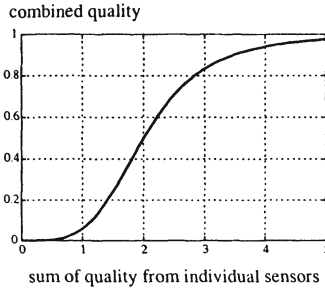


Figure 11.17. Non-linear adhesion function: $y = x^4 / (2^4 + x^4)$



Figure 11.18. Overlapping measurements from the same sensor

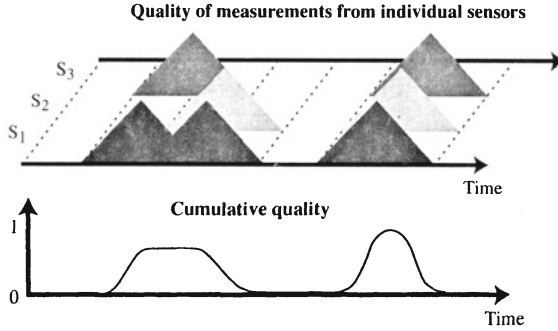


Figure 11.19. Overlapping measurements from different sensors

The adhesion function can be applied for every instant (over some appropriate time period), giving an overall time-quality curve, as illustrated in Figure 11.19.

For *multiple* targets, the adhesion function as described above applies to each single-target metric separately: for each target and for each instant, the effective metrics (as determined by the persistence function) of all of the measurements are summed and a combined quality computed.

Let $G \equiv \bigcup_{i \in \text{dom}(\vec{w})} \text{dom}(w_i)$ be the set of all targets that are represented in any world state in the trajectory. Then for each $g \in G$:

$$\phi_{\vec{w}g}(\vec{m}, t) \equiv \alpha([i \in \text{dom}(\vec{m}) \rightarrow \phi_{w_i g}(m_i) \cdot \pi(t - t_i)]) \quad (11.10)$$

where $\phi_{wg}(m)$ denotes the (peak) quality of a measurement m for the given target in the given world state (see Equations 11.4 & 11.5), and $\text{dom}(\vec{m}) \equiv \text{dom}(\vec{w}) \equiv \text{dom}(\vec{t})$ by definition. An expression of the form $[i \in D \rightarrow f_i]$ denotes a vector having one element for each $i \in D$, whose value is f_i ; the order of the elements is not important (in other words, it denotes a finite map rather than a vector) because α is not dependent on the order (by assumption).

It is assumed that target information represented in separate measurements can be correlated; i.e., that if one measurement has information regarding some target that it labels A and another measurement has information regarding a target that is also labeled A , then both measurements actually concern the same real-world target.

In this section, all targets arise from the evolution of a common (global) state so target correlation is moot. However, when state information is distributed/localized, correlation may need to be taken into account.

The following assumption, regarding multiple measurements that overlap in time, has so far been made tacitly:

ASSUMPTION 5 (NON-INTERFERING SENSORS) *The results of a measurement made by one sensor are not affected by any measurements being made simultaneously by other sensors.*

That is, although the *usefulness* of a measurement may be affected by what other measurements are being taken simultaneously, the *raw signals* emitted/received by the sensors do not interfere.

In practice, if two sensors are known to interfere, a mutual-exclusion restriction can be imposed — restrictions are discussed below.

3.9.3 Overall Quality. To allow different proposed vectors of measurements to be compared, a single value can be derived representing the overall quality that is expected to accrue from the vector of measurements. One possible form for the overall quality metric is the sum over targets of the time-average of the instantaneous, single-target metrics:

$$\psi_{\vec{w}}(\vec{m}) \equiv \sum_{g \in G} \int_T \phi_{wg}(\vec{m}, t) dt \quad (11.11)$$

where T is some reasonable time period encompassing $\vec{t}$.

3.9.4 Scaling and Adding Mappings. In the preceding sections, it was assumed that multiple-target metrics, which are structured values, can be scaled and added. This section defines these operations.

A single-state metric is a map from targets to single-target metrics. Targets can be represented by uninterpreted symbols (identifiers) and a single-target

metric can be taken to be a real number. Thus, the type of a single-state metric is $\text{target} \rightarrow \text{real}$.

To scale a single-state metric, each single-target metric is scaled uniformly:

$$s \{g \in G \rightarrow v_k\} \equiv \{g \in G \rightarrow sv_k\}$$

where s is a scalar value.

To add two single-state metrics, terms for targets that occur in both metrics are added, and terms that occur in only one of the metrics are carried into the sum unchanged.

$$\{g \in G_1 \rightarrow v_{1g}\} + \{g \in G_2 \rightarrow v_{2g}\} \equiv \{g \in G_1 \cup G_2 \rightarrow v_{12g}\}$$

where v_{12g} is defined by cases:

$$v_{12g} \equiv \begin{cases} v_{1g} + v_{2g} & \text{if } g \in G_1 \wedge g \in G_2 \\ v_{1g} & \text{if } g \in G_1 \wedge g \notin G_2 \\ v_{2g} & \text{if } g \notin G_1 \wedge g \in G_2 \end{cases}.$$

3.10 Measurement Feasibility

In the preceding sections, the quality of a proposed vector of measurements was considered under the assumption that the network could actually take all of the measurements. However, individual sensors and collections of sensors must observe some *restrictions* that limit the space of *feasible* vectors of measurements. In the radar example, two restrictions are: (i) an emitter must be activated at least two seconds before a measurement can be taken; (ii) on a given radar, no more than one detector can be sampled at any time.

The emitter stability restriction relates a measurement to an *action* that is not a measurement, namely turning on an emitter. In general, restrictions can involve any type of action the sensor network can execute, which may be, e.g., measurements or changes to the hardware state made in preparation for measurements. Consequently, the coordination mechanism must generate a *schedule* which assigns *actions* to the network's resources for execution over specified time periods. Figure 11.20 shows an example schedule.

Restrictions can be formulated as constraints in the sense of score functions described earlier: for example, violation of a restriction may generate a score whose value is negative, compared with constraints formulating target tracking that generate positive scores. Given a schedule of actions A and a set of restrictions $\{p_i\}$, the total violation penalty is the sum of the penalties for the individual restrictions: $P(A) \equiv \sum_i p_i(A)$. (Note that each constraint can incorporate an appropriate weighting factor.)

Alternatively, restrictions can be accounted for by eliminating from a schedule all actions that violate restrictions, thus leaving behind a feasible, residual

<i>Period</i>	<i>Resource</i>	<i>Action</i>
0.00–0.01	radar 1	activate emitters 0 & 1
2.00–2.60	radar 1	sample detector 0
2.00–2.01	radar 2	activate emitter 2
2.60–3.20	radar 1	sample detector 1
3.20–3.21	radar 1	deactivate emitters 0 & 1
4.00–4.60	radar 2	sample detector 2

Figure 11.20. Example of a schedule

schedule. Indeed, it may be useful to treat restrictions in both ways: during the schedule search phase, restrictions may be treated as negative score functions to help guide the search towards high-quality schedules, but just prior to schedule execution they are treated as filters to remove infeasible actions.

3.11 Overall Quality (including Operational Cost)

Given a schedule of actions, it is typically straightforward to assess the cost of executing the schedule. For the radar example, the main cost is the energy consumed by emitters: when an emitter is active, it consumes energy at a constant rate, regardless of whether or not its detector is being sampled. This energy cost can be computed from the times at which the emitters are activated and deactivated.

The details of how to assess costs do tend to be application specific, so they will not be considered further. However, it is assumed that, for a schedule A , the costs $C(A)$ can be assessed on a scale that is commensurate with the measurement quality $M_{\omega\epsilon}(A)$ and the penalty metric $P(A)$ so that an overall quality metric $Q(A)$ can be determined. For this treatment, a simple linear combination is assumed:

$$Q_{\omega\epsilon}(A) \equiv M_{\omega\epsilon}(A) - P(A) - C(A). \quad (11.12)$$

An action schedule contains measurement and non-measurement actions. To determine its measurement quality, the non-measurement actions are expunged to give a vector of measurements $\vec{m}$ and $\Psi_{\Omega}(\vec{m})$ computed (see Section 11.3.8).

4. Peer-to-Peer Sensor Coordination Algorithm

Equation 11.12 defines a metric function $Q_{\omega\epsilon}$ for assessing, with respect to an existing world estimate and model of evolution, the quality of a proposed schedule of actions to be executed by a sensor network.

The responsibility of the network's coordination mechanism may be loosely defined as determining a schedule that optimizes $Q_{\omega\epsilon}$. Given that a sensor network is subject to changing circumstances (such as hardware failing and

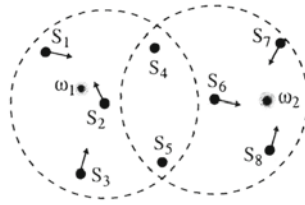


Figure 11.21. Different fusion nodes receive different measurements

targets not following predictions), a *real-time* coordination mechanism must continually update the network's schedule to maintain optimality as it learns new information about the real world.

The following sections show how the quality metric can be used by the peer-to-peer optimization algorithm to achieve distributed coordination of the sensors.

4.1 Local World Estimates

The coordination mechanism is based on knowledge of the real world, as represented by a world estimate. In order to achieve a scalable, distributed coordination mechanism, it is necessary to distribute the world estimate as a collection of local world estimates and to maintain each local world estimate using only local measurements.

In this treatment, each sensor S_j is uniquely, tightly coupled with a *fusion* or *tracking node* that maintains a local world estimate ω^j that represents information about nearby targets. Each fusion node maintains its estimate based on readings acquired by its own sensor and on readings received from nearby sensors — each fusion node periodically broadcasts its sensor's measurements.

Since any two fusion nodes may be within communication range of different sets of sensors, the readings that they receive may differ, and consequently the local estimates they compute may differ. For example, Figure 11.21 illustrates two target estimates, ω_1 computed by sensor S_2 's fusion node and based on measurements from S_1 , S_2 and S_3 , and estimate ω_2 computed by sensor S_6 's fusion node and based on measurements from S_6 , S_7 and S_8 . However, the differences between estimates computed by *nearby* fusion nodes are likely to be small, and not important as regards coordinating the sensors.

Just as the coordination mechanism continually updates sensor schedules, so too each fusion node continually updates its local estimate as it receives measurements from its own sensor and from nearby sensors. It is assumed that the implementation of this process is straightforward and it is not considered further here.

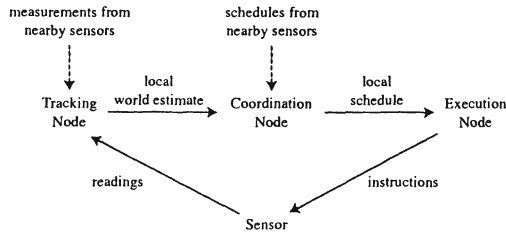


Figure 11.22. Components coupled with a sensor

4.2 Local Schedules

Each sensor is uniquely, tightly coupled with a *coordination node* that maintains a *local* schedule of actions for the sensor to execute. That is, once a local schedule has been determined, it can be executed without further coordination with other sensors — all inter-sensor collaboration on measurements occurs through each sensor independently executing its own actions at the correct times (which it can accomplish using its local, synchronized clock).

This independent execution is vital to circumventing communication latency: individual sensor actions occur too frequently for them to be initiated by some mechanism that is not resident on the sensor itself. Of course, inter-sensor communication is still required to determine the local schedules, as discussed in the next section.

Figure 11.22 illustrates the interactions of various components associated with each sensor.

4.3 Local Schedule Quality Metrics

Since each local schedule A^j contains actions for only sensor S_j , the global schedule can, conceptually, be formed as the union of the local schedules: $A \equiv \bigcup_j A^j$. Consequently, the global schedule quality metric (see Equation 11.12) can be easily redefined in terms of local schedules.

However, the runtime *computation* of the metric in a distributed environment would require extensive communication and is not practical for a real-time application. Instead, coordination nodes compute local metrics that collectively substitute for the global metric.

ASSUMPTION 6 (ENCOMPASSING COMMUNICATION) *The collaboration graph is a sub-graph of the communication graph.*

In other words, if two sensors are capable of usefully collaborating on scanning a target, then they are capable of communicating.³

If every coordination node broadcasts its sensor's schedule then each coordination node will know the schedules of all sensors with which direct collaboration may be useful. A local quality metric can be based on this knowledge, as follows.

Let H^j be the set of (other) sensors with which sensor j can communicate. Let A^{jk} (where $k \in H^j$) be j 's copy of k 's schedule. Then the fragment of the global schedule that is known to j is $\tilde{A}^j \equiv A^j \cup \bigcup_k A^{jk}$ and j 's local quality metric is $Q_{\omega^j \epsilon}^j \equiv Q_{\omega^j \epsilon}(\tilde{A}^j)$. Note that the metric is with respect to the local world estimate ω^j .

The global quality metric can, presumably, be expressed as a function of the local quality metrics (with appropriate account taken of the fact that each measurement may be incorporated into multiple local metrics). In general, the relationship between the global and local metrics is not simple. For example, consider a process in which each coordination node simultaneously changes its local schedule so as to optimize its local metric, and assume that a stable set of local schedules is found; i.e., for every j , A^j optimizes Q^j given A^{jk} ($k \in H^j$). Even though every local schedule is, in a sense, locally optimal, there is no guarantee that the combined schedule optimizes the global metric.

Nevertheless, this is essentially the process that is followed by the distributed coordination mechanism described below. The fact that global optimality is not guaranteed is viewed as a sacrifice that is required to achieve scalability and real-time responsiveness. Whether or not this sacrifice is worthwhile can only be determined in the context of a particular application.

4.4 Distributed Coordination Mechanism

Given the comments of the preceding sections, the objective of a distributed coordination mechanism is to determine a set of local schedules A^j such that each local quality metric Q^j is optimized by A^j given j 's copies A^{jk} ($k \in H^j$) of the local schedules of the sensors with which j might collaborate. In the following sections, the details of the implementation of the distributed coordination algorithm for the challenge problem are presented.

4.4.1 Target Models. The targets are assumed to be deterministic and to move with constant velocities and, for coordination, it is assumed that the

³For flat geographies and for homogeneous sensors, the communication assumption is satisfied if the communication range is twice the useful sensing range. To be strictly accurate, the useful sensing range should be extended with the largest distance a target could move over the time span of a schedule, since a target may move from this extended region into the collaborative sensing region.

number of targets n is fixed over a single coordination step (this assumption is not made for data fusion).

Each world state represents each target's position and velocity:

$w \equiv \{g \in \{1, \dots, n\} \rightarrow \langle p_g, v_g \rangle\}$. For the proximate metric, the form (Equation 11.8) based on the most likely world trajectory w^* is used: $\Psi_\Omega(\vec{m}) \equiv \psi_{\vec{w}^*}(\vec{m}) \cdot \Omega(\vec{w}^*)$.

For a fixed number of multiple, independent targets, the most likely world trajectory, on a vector of times $\vec{t}$, is the combination of the most likely trajectories for each target. Since the targets are taken to be deterministic and have constant velocity, each target's most likely trajectory is given by $w_i^* = \{g \in \{1, \dots, n\} \rightarrow \langle \bar{p}_g + \bar{v}_g \cdot (t_i - t_0), \bar{v}_g \rangle\}$ where $\bar{p}_g, \bar{v}_g$ are the means of target g 's position and velocity components of the probability distribution at the start of the coordination step, at time t_0 .

Under this formulation, each target can be considered to generate a constraint that connects all sensors within scanning range of the target and that has a score function defined below. This makes the challenge problem precisely a distributed constraint optimization problem.

4.4.2 Schedules. A schedule A^j for a single sensor j has two components: a measurement schedule A_m^j and a non-measurement action schedule A_a^j . A schedule is a map from *time slots* to measurements or actions.

- Each time slot begins at an integral multiple of 0.6 seconds (relative to some arbitrary time origin) and lasts for 0.6 seconds.
- The real-world time corresponding to the mid-point of time slot k is denoted as $T(k)$.
- In the measurement schedule, each time slot may contain information for at most one measurement.
- An amplitude-only measurement requires a single time slot; an amplitude-and-frequency measurement requires three successive time slots.
- In the action schedule, each time slot may contain information for multiple actions.
- A single non-measurement action requires a negligible amount of time to execute — all of the non-measurement actions associated with a time slot are thought of as occurring at the start of the time slot.

4.4.3 Local Metrics. The task of a single coordination node is to assign measurements and actions to its schedule's time slots over some reasonable time period, say for the next 12.6 seconds (which corresponds to 21 time slots). The assignments should optimize the overall local quality metric, given the current local world estimate and current copies of other sensors' schedules.

Figure 11.4.4.3 summarizes the main steps in computing (an approximation to) a local measurement quality metric using the most-likely trajectory. To

compute the measurement quality, the ‘triangular’ persistence function shown in Figure 11.16 and the adhesion function shown in Figure 11.17 are used. The non-zero region of the persistence function is taken to be 2 seconds, so the equation of the function is $\pi(\Delta t) \equiv \max[0, 1 - |\Delta t|]$. The integral of the quality over time in Equation 11.11 is approximated by a discrete sum over the time slots. The vector of times $\vec{t}$ on which the most-likely trajectory is founded is given by $t_k \equiv T_m(k)$.

The total local penalty metric can be computed by summing the penalties for each individual restriction. Efficient methods for computing the penalty for a restriction depend on the form of the restriction.

The cost of executing a schedule can be determined using a simulation of the schedule’s effect on the sensor (as discussed above). For example, every time the simulation determines that an emitter is deactivated, an energy cost can be assessed based on when the emitter was last activated.

Note: in order to produce an accurate simulation, the state of the sensor at the start of the coordination step must be known. This can be determined either by querying the hardware for its true state, or querying some model maintained by the execution node.

4.4.4 Search. The size of the search space for even a local schedule is large. Neglecting everything except amplitude-only measurements, there are four possible choices for each time slot: a measurement using one of the three detectors or no measurement. If the schedule extends over 21 time slots, the number of combinations is $4^{21} \approx 4 \times 10^{12}$. This is probably too large for a generative search technique even if pruning is used.

Consequently, a transformational search algorithm is used. An initial schedule is computed by independently choosing for each time slot an amplitude-only measurement that maximizes the single-measurement quality — collaboration, feasibility and operational costs are ignored. This schedule is then iteratively improved using hill-climbing on the full quality metric via transformations that include:

- Removing a measurement from a time slot, so that no measurement is performed.
- Changing the detector that is sampled in a time slot.
- Changing three successive amplitude-only measurements into an amplitude-and-frequency measurement.
- Inserting and removing emitter activation and deactivation actions.

(Of course, finite differencing techniques should be applied to the computation of the quality metric described in the preceding section, so that the quality can be cheaply recomputed after transformation.)

1 Compute target information.

For each target g compute the mean position $\bar{p}_g$ and velocity $\bar{v}_g$ according to the initial estimate ω (for time t_0).

2 Compute single-measurement metrics.

For each time slot k in A_m :

- (a) Determine the expected position of each target: $p_g(k) \equiv \bar{p}_g + \bar{v}_g \cdot (T_m(k) - t_0)$.
- (b) For each target g and each measurement m whose mid-point occurs in this time slot in the local schedule or in any of the schedules of other sensors, determine the single-target quality metric that would be awarded for that sensor if the target were present alone. For example:

$$\phi_{mg}(k) \equiv \log_2 \left(1 + \frac{\max[0, m(R, \theta) - m_b]}{m_{\max} - m_b} \right) \quad (11.13)$$

where R, θ are the distance between the sensor and the target and the angle between the emitter's mid-beam and the target, $m(R, \theta)$ computes the expected signal strength reflected from the target (Equation 11.1), m_b is a typical background noise level for the sensor that is to take the measurement, and $m_{\max}$ is the largest possible reading for that sensor.

- (c) Determine the multiple-target metric for the time slot using Equation 11.5.

3 Compute multiple-measurement metrics.

For each time slot k :

- (a) Determine all measurements whose persistence function have a non-zero value at the time slot's mid-point (i.e., all measurements that occur within 1 second of the mid-point).
- (b) For each of these measurements, compute the effective quality metric at the time slot's mid-point by scaling the measurement's peak metric according to the persistence function (Equation 11.9).
- (c) Then combine all of the effective metrics according to the adhesion function, to give a single quality metric for each target (Equation 11.10).

4 Compute an overall quality metric.

Sum the single-time-slot, multiple-target metrics (scaled by 0.6 seconds) over all time slots and targets to give one value that represents the overall quality of the schedule (see Equation 11.11).

Figure 11.23. Algorithm for computing a local metric on measurement quality

Given that the coordination mechanism is subject to real-time constraints, it may happen that it cannot complete its search in time. Fortunately, a feasible schedule can be quickly extracted for execution. The resulting schedule, of course, is probably not optimal (even in a local sense). However, as the execution node begins processing the schedule, the coordination node can continue to improve it. In particular, there is still time to change actions that are not scheduled to begin for another few seconds: consequently, the search should first focus on the initial section of the schedule and improve later sections only when the initial section has attained sufficiently high quality.

If the transformational search algorithm ranks candidate transformations according to potential quality improvement, then a simple way to assign more attention to the initial section of the schedule is to magnify its contribution to the quality metric. This has the advantage of not *entirely* neglecting later parts of the schedule.

Of course, it may happen that even with its ongoing, anytime search approach, the coordination node is simply incapable of producing schedules of sufficiently high quality in the time and with the computational power available. To determine if this is so, a detailed analysis of the probability distribution of schedules and consequent computational costs associated with the search and metric algorithms may be performed. Alternatively, the approach can be validated experimentally.

4.5 Experimental Results

Figure 11.24 shows tracking results produced by the P2P optimization algorithm using the challenge problem simulator. There are 8 sensors, indicated by the groups of triangles (indicating the 3 emitter-detector pairs per sensor). There are two targets, moving in oval tracks, indicated by the unbroken lines, around a room of dimensions 40 units by 40 units.

The black dots represent typical output from a tracking node: each dot is an estimate of a target's position. The error in a position estimate $\vec{p}$ at time t can be computed as the distance between $\vec{p}$ and the true target position at time t . The track quality can be measured as the root-mean-square of the single-estimate errors.

For a single target (not illustrated), r.m.s. errors of about 0.6 units are obtained; for two targets, about 0.9 units. These values are close to the limit (about 0.5 units) of what the tracker is expected to be able to achieve, given its approximation techniques, its model of the targets, and (simulated) sensor noise. They also compare well with errors obtained using purely random choice of sectors to be measured, namely about 2.2 units for a single target.

The mean power usage is about 50% (meaning that, on average, half the emitters are on at any given time). This is slightly higher than would be hoped

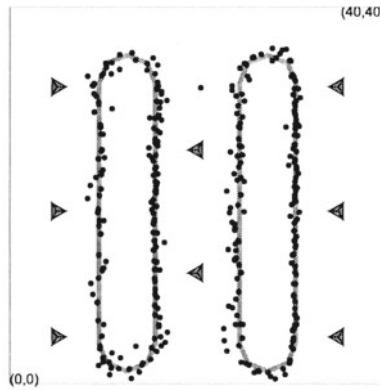


Figure 11.24. Tracking two targets

for, especially for a single target, for which it should be possible to reduce the power usage to around 33% (meaning that only one emitter is active on each sensor, on average).

5. Related Work

Most work on distributed constraint satisfaction or optimization addresses algorithms in the classes of simulated annealing or parallel branch-and-bound, in which *the search space* is distributed over some network of nodes and in which each node contains a complete solution to the problem.

In contrast, in the algorithm considered here, a single solution is distributed over the network of nodes/variables, so that each node contains only a small fragment of a complete solution. This latter approach seems better suited to real-time control problems, in which communication latency essentially prohibits the gathering of information into a single node.

Other work on this latter form of distribution has been carried out by Yokoo *et al.* [Yokoo 1998]. A deterministic version of the FP algorithm considered here was published in [Fabiunke 1999] — its performance is compared with the FP algorithm in [Fitzpatrick 2001]. However, these and other works seem to emphasize the long-term convergence properties of algorithms, whereas the property considered most important here is the short-term improvement, since that relates directly to real-time responsiveness.

6. Conclusions

The framework of distributed constraint optimization seems well suited to many distributed control problems in which the problem size and inherent la-

tency preclude centralized solutions. The DCO framework formulates such problems in terms of localized functions that can be locally optimized.

The peer-to-peer algorithm presented here is simple but has proven to be surprisingly effective. Experiments with abstract problems have shown it to be adaptive, scalable, cost-effective and robust. Experiments with the challenge problem simulator are best considered as proof-of-concept demonstrations that the algorithm can be used for more complex problems.

A characteristic aspect of the P2P algorithm is its *anarchic* nature — it lacks any non-local control authority. It may thus be expected to be very robust and simple to implement, but it remains to be seen how large is the class of practical problems that can be addressed in this way.

Much work remains to be done on the P2P optimization algorithm: in particular, analytical results are almost entirely absent for such issues as stability and rates of convergence.

7. Acknowledgments

This work is sponsored in part by DARPA through the ‘Autonomous Negotiating Teams’ program under contract #F30602-00-C-0014, monitored by the Air Force Research Laboratory. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the Defense Advanced Research Projects Agency or the U.S. Government.

References

- Culberson, Joseph (1992). *Iterated Greedy Graph Coloring and the Difficulty Landscape*, Technical Report TR 92-07, Department of Computing Science, The University of Alberta, Edmonton, Alberta, Canada, June 1992.
- Fabiunke, Marko (1999). *Parallel Distributed Constraint Satisfaction*, Proceedings of the International Conference on Parallel and Distributed Processing Techniques and Applications (PDPTA’99), pp. 1585-1591.
- Fitzpatrick, Stephen, & Meertens, Lambert (2001). *An Experimental Assessment of a Stochastic, Anytime, Decentralized, Soft Colourer for Sparse Graphs*, 1st Symposium on Stochastic Algorithms: Foundations and Applications, Lecture Notes in Computer Science 2264, Kathleen Steinhöfel (Ed.), Springer-Verlag ISBN 3-540-43025-3, pp. 49-64.
- Lewandowski, Gary, & Condon, Anne (1996). *Experiments with Parallel Graph Coloring Heuristics in Cliques, Coloring and Satisfiability*, DIMACS Series in Discrete Mathematics and Theoretical Computer Science, vol. 26, American Mathematical Society, 1996, pp. 309-334.
- Meertens, Lambert, and Fitzpatrick, Stephen (2001). *Peer-to-Peer Coordination of Autonomous Sensors in High-Latency Networks using Distributed*

Scheduling and Data Fusion, Technical Report KES.U.01.09, December 2001, Kestrel Institute, Palo Alto, California.

Yokoo, Makoto, Durfree, Edmund H., Ishida, Toru & Kuwabara, Kazuhiro (1992). *The Distributed Constraint Satisfaction Problem: Formalization and Algorithms*, IEEE Trans. on Knowledge and Data Engineering, vol. 10, no. 5, September/October 1998.